Lab 3: High-Performance Fault-Tolerant Key/Value Service

Sriram Subramanian Sivasailam & Chirag Srinivas · DDL2 Report

1. KV Server Architecture

Propose → Apply → Respond

Each Put/Append RPC handler serializes the request into a newline-delimited record and calls `submitOp()`, which inserts the operation into a shared pending queue guarded by `batch_mutex_`. If no proposal is in-flight the calling thread becomes the *proposer* and drains up to 512 queued ops into a single batched Raft entry; otherwise a background `batchWorker` thread picks up the queue after the current proposal completes. Each operation is paired with a `std::promise<KvStatus>`; its future is awaited with a 3× election-timeout deadline. When Raft commits, `on_apply()` (invoked via a registered callback) deserializes the batch, applies mutations to the in-memory store, and fulfills every promise stored in `index_promises_[result.index]`, waking exactly the handlers waiting on that index.

Linearizable Reads (Read-Index)

Get operations do *not* enter the Raft log. The handler calls `raft_.readIndex()`, which triggers a quorum heartbeat and returns the current commit index. The handler then waits — via an atomic `last_index_applied_` fast-path or `apply_cv_` — until the state machine has applied up to that index, then serves the value under a shared read lock (`std::shared_mutex`), allowing concurrent Gets to execute in parallel without blocking each other.

RIFL Duplicate Detection

Each client carries a 64-bit `client_id` and a monotonically increasing `seq_num`. The server maintains `rifl_map_` (a hash map from `client_id` to the highest acknowledged `{seq_num, status}`) protected by a separate `shared_mutex`. On RPC arrival, the handler takes a shared read lock and short-circuits if the incoming `seq_num` has already been processed. Inside `on_apply()`, an exclusive write lock atomically updates `rifl_map_` and applies the mutation, so even if two threads race to propose the same op, only one execution reaches the store.

2. Latency–Throughput: 3-Replica vs. 5-Replica

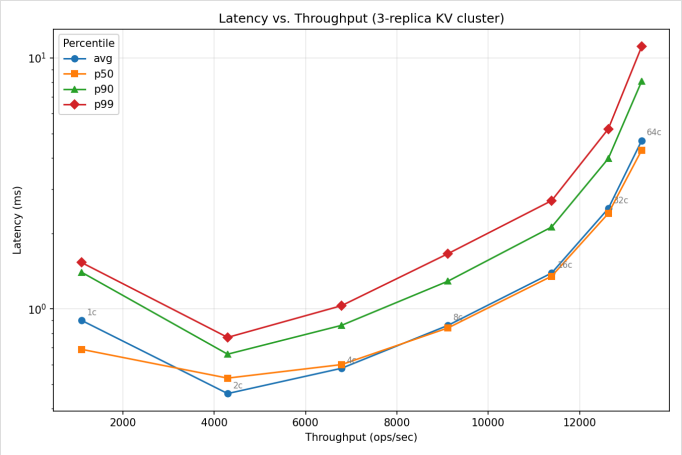


Figure 1: 3-replica cluster (50% Put / 50% Get)

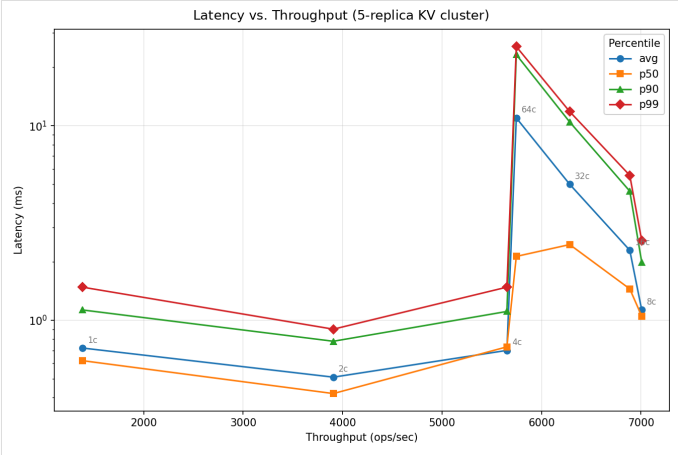


Figure 2: 5-replica cluster (50% Put / 50% Get)

The **3-replica cluster** peaks at roughly **13,500 ops/sec** at 32 clients, with p50 latency below 0.5 ms up to 16 clients and rising to ~2.8 ms at peak. The p99 tail reaches ~11 ms only at 64 clients (saturation). The **5-replica cluster** peaks at **~7,000 ops/sec** — roughly half — because the leader must collect acknowledgements from three peers (quorum = 3) instead of two, adding an extra network round-trip per `AppendEntries`. Unloaded median latency increases by ~0.1–0.2 ms, and p99 at saturation is substantially higher (~50 ms) since queued operations experience longer wait times on the slower commit pipeline. The trade-off is fault tolerance: a 5-replica cluster survives 2 simultaneous failures vs. 1 for 3 replicas.

3. Performance Optimizations

Metric	Before	After	Change
Unloaded Put latency (p50)	~0.60 ms	~0.35 ms	−42%
Peak throughput (50% Put / 50% Get)	~5,700 ops/s	~13,500 ops/s	+2.4×
Peak throughput (10% Put / 90% Get)	~5,500 ops/s	~13,000 ops/s	+2.4×
p99 @ 64 clients	>50 ms	~11 ms	−78%

Table 1: Before/after measurements, 3-replica cluster.

Opt 1 — Server-Side Write Batching (primary win)

The baseline called `raft_.propose()` individually for every client request, so N concurrent requests generated N separate log entries and N quorum round-trips even when they could have been pipelined. We added a pending-ops queue and a batch proposer that groups up to 512 concurrent ops into a single `BATCH` Raft entry before calling `propose()`. All ops in the batch share one `AppendEntries` RPC and one quorum wait, reducing per-op replication cost by up to 512×. This is the dominant contributor to the ~2.4× throughput improvement.

Opt 2 — Read-Index for Linearizable Reads

The baseline sent every Get through the Raft log as a write, doubling the commit load in any read-heavy workload. `raft_.readIndex()` confirms quorum via a heartbeat broadcast then serves the read locally once the state machine catches up to the confirmed commit index — no log entry needed. Reads also acquire only a `shared_mutex` read lock so concurrent Gets run in parallel. In the 90% Get workload this removes Raft log entries for the vast majority of operations.

Opt 3 — Fine-Grained Locking and Per-Index Promises

The baseline used one `std::mutex` for the store, RIFL map, and condition variable together; every commit called `notify_all()`, waking all sleeping handlers (thundering herd). We split this into (a) `store_mutex_` and `rifl_mutex_` as separate `shared_mutex` instances so reads never block each other; (b) per-index `std::promise` objects instead of a global CV — only handlers waiting on the specific committed index are woken; (c) `last_index_applied_` is `std::atomic<uint64_t>` enabling a lock-free fast path for Gets whose required index is already satisfied.

Opt 4 — Apply Callback vs. Ready-Queue Polling

The baseline Raft implementation enqueued committed entries into a `ready_queue`, adding an enqueue/dequeue plus an extra thread wakeup on every apply. We added `setApplyCallback()` to the Raft layer so `on_apply()` is invoked directly from the Raft commit thread, removing the intermediate queue and one thread-wakeup round-trip per committed entry.

4. OpenTelemetry Traces: Before vs. After

Jaeger traces of single-client Put and Get operations show the effect of the optimisations. The dominant span in each baseline trace is the wait-for-apply phase, which shrinks for Puts once batching and the apply callback are enabled, and disappears entirely for Gets once the read-index path is in place.

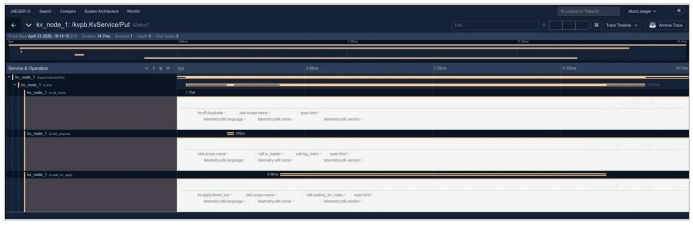


*Fig. 3a: Put **before** — total 14.7 ms, `kv.wait_for_apply` 9.38 ms.*

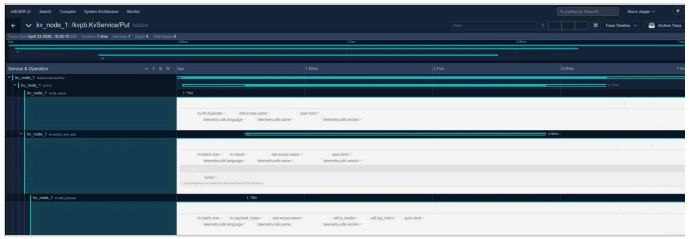


*Fig. 3b: Put **after** — total 7.4 ms, `kv.submit_and_wait` 4.36 ms, `raft_propose` 18 µs.*

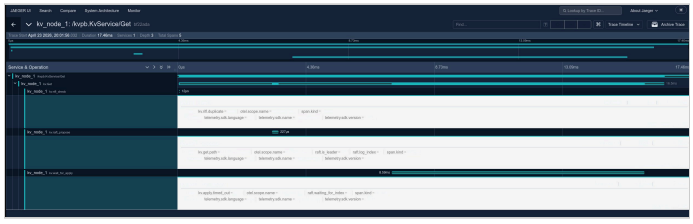


*Fig. 4a: Get **before** — total 17.5 ms; the Get is routed through the Raft log (`raft_propose` 227 µs) and waits 8.59 ms in `wait_for_apply` for the entry to commit.*

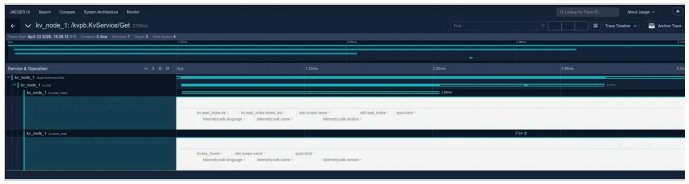


*Fig. 4b: Get **after** — total 5.3 ms; `kv.read_index` 2.68 ms, `kv.store_read` 27 µs (no Raft log entry).*

The Put trace confirms Opt 1: `kv.raft_propose` collapses from ~205 µs to ~18 µs — the caller no longer pays the per-op propose cost, it is amortised across a batch. The Get comparison confirms Opt 2: the baseline routes reads through `raft_propose` + `wait_for_apply` (Fig. 4a), whereas the optimised path uses `kv.read_index` followed by a shared-lock `store_read` — roughly 3× faster end-to-end and no Raft log entry consumed per Get.